

Tecnológico de Costa Rica

Escuela de Computadores

Disney's Epic Adventure - Imaginary Battle

Documento Técnico

Estudiante: Andrés Hernández García

Carné: 2021138201

Curso: Introducción a la Programación

Profesor: Elbio Alberto Ramírez Trejos

Fecha: 9 de Mayo 2026

Índice

Índice.....	2
Palabras clave y Acrónimos.....	3
Introducción.....	3
Marco Teórico.....	4
Descripción del Programa.....	4
Diagrama de flujo.....	5
Resultados Obtenidos.....	6
Descripción de las Funciones Principales... ..	6
Descripción de la Interfaz Gráfica.....	7
Conclusiones.....	8
Recomendaciones.....	8
Bibliografía.....	8

Palabras clave y Acrónimos

Palabras clave: juego por turnos, interfaz gráfica, recursividad, Python, Tkinter, batalla.

Acrónimos:

GUI: Graphical User Interface (Interfaz Gráfica de Usuario)

HP: Hit Points (Puntos de Vida)

ATK: Attack (Ataque)

DEF: Defense (Defensa)

KO: Knock Out

Introducción

El presente documento técnico describe el desarrollo del juego Disney's Epic Adventure, una aplicación de escritorio desarrollada en Python utilizando la librería Tkinter para la interfaz gráfica. El juego simula batallas por turnos entre personajes, inspirándose en la dinámica de juegos como Pokémon y Final Fantasy.

El jugador actúa como un Guardián de las Historias cuya misión es derrotar a 5 Hollows ubicados en diferentes tierras. Para lograrlo debe seleccionar 3 personajes de una lista de 15 disponibles y enfrentarse a los Hollows en combates por turnos.

El proyecto fue desarrollado como parte del curso de Programación del Tecnológico de Costa Rica, con el objetivo de aplicar conceptos como recursividad, estructuras de datos, programación orientada a objetos e interfaces gráficas.

Marco Teórico

El sistema fue desarrollado utilizando las siguientes tecnologías y conceptos:

Python 3.14 Lenguaje de programación de alto nivel seleccionado por su facilidad de aprendizaje, legibilidad y amplio ecosistema de bibliotecas. Todo el juego fue desarrollado completamente en Python.

Tkinter Librería estándar de Python para la creación de interfaces gráficas de usuario. Se utilizó para construir todas las ventanas, botones, etiquetas e imágenes del juego.

Pillow (PIL) Librería de procesamiento de imágenes para Python. Se utilizó para cargar, redimensionar y mostrar las imágenes de los personajes y fondos de batalla.

Programación Orientada a Objetos El juego utiliza clases para representar los personajes y las pantallas. La clase principal es **Personaje**, que contiene los atributos vida, ataque y defensa, y los métodos para calcular daño y recibir daño.

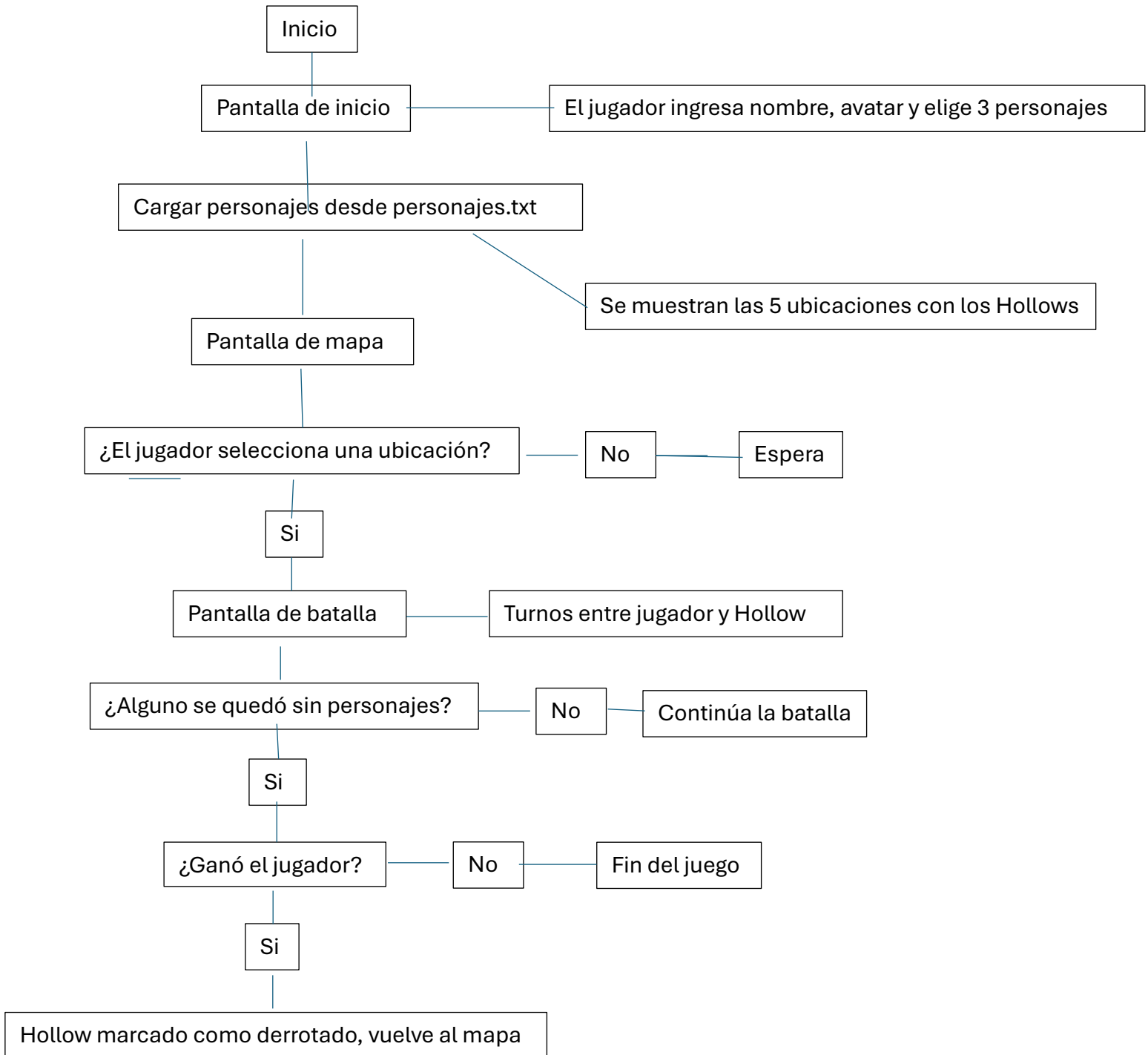
Rekursividad Se utilizó recursividad en las funciones `verificar_fin_batalla_hollow` y `verificar_fin_batalla_jugador`, las cuales recorren la lista de personajes disponibles llamándose a sí mismas hasta encontrar un personaje disponible o determinar que todos están KO.

Archivos de texto Los 15 personajes del juego están almacenados en un archivo de texto llamado `personajes.txt`, el cual es leído al iniciar el programa mediante la función `cargar_personajes`.

Descripción del Programa

El programa inicia con la pantalla de inicio donde el jugador ingresa su nombre, selecciona su avatar y elige 3 personajes de una lista de 15. Al presionar **INICIAR** se muestra el mapa con las 5 ubicaciones donde se encuentran los Hollows. El jugador se mueve entre ubicaciones y al entrar a una inicia una batalla por turnos hasta derrotar al Hollow o ser derrotado.

Diagrama de flujo general del sistema:



Resultados Obtenidos

Descripción de las Funciones Principales

cargar_personajes() — Lee el archivo personajes.txt línea por línea y crea un objeto Personaje por cada línea. Entradas: archivo de texto. Salidas: lista de objetos Personaje.

calcular_danio(defensor) — Calcula el daño que un personaje le hace a otro usando la fórmula ATK del atacante menos DEF del defensor, con un mínimo de 1. Entradas: objeto Personaje defensor. Salidas: valor entero del daño.

recibir_danio(danio) — Resta el daño recibido a la vida actual del personaje. Si la vida llega a 0 el personaje queda marcado como KO. Entradas: valor entero del daño. Salidas: actualiza el estado del personaje.

restaurar_vida() — Restaura la vida del personaje al máximo cuando un entrenador se adueña de él. Entradas: ninguna. Salidas: vida restaurada al máximo.

verificar_fin_batalla_hollow(indice) — Función recursiva que recorre la lista de personajes del Hollow buscando uno disponible. Si todos están KO declara la victoria del jugador. Entradas: índice entero. Salidas: continúa la batalla o la termina.

verificar_fin_batalla_jugador(indice) — Función recursiva que recorre la lista de personajes del jugador buscando uno disponible. Si todos están KO declara la derrota del jugador. Entradas: índice entero. Salidas: continúa la batalla o la termina.

turno_hollow() — Ejecuta el turno del Hollow de forma completamente aleatoria, eligiendo entre atacar o cambiar de personaje. Entradas: ninguna. Salidas: actualiza el estado de la batalla.

Descripción de la Interfaz Gráfica

El juego cuenta con tres pantallas principales desarrolladas con Tkinter. La pantalla de inicio permite al jugador ingresar su nombre, seleccionar su avatar y elegir 3 personajes de una lista de 15. La pantalla de mapa muestra las 5 ubicaciones con sus Hollows y permite al jugador moverse entre ellas. La pantalla de batalla muestra los personajes enfrentados con sus imágenes, una barra de log donde se registran los eventos de la batalla y botones para atacar o cambiar de personaje. Cada batalla tiene un fondo diferente según la ubicación donde se pelea.

Conclusiones

El sistema desarrollado cumple con todos los requerimientos establecidos por el profesor. Se logró implementar un juego de batalla por turnos funcional con interfaz gráfica usando Tkinter, personajes cargados desde un archivo de texto, recursividad en la lógica de batalla y un sistema de mapa con 5 ubicaciones.

El uso de programación orientada a objetos permitió organizar el código de forma clara y modular, separando la lógica del juego en archivos independientes para cada pantalla.

La recursividad fue aplicada correctamente en las funciones de verificación de fin de batalla, cumpliendo con el requisito obligatorio del proyecto.

Recomendaciones

Como mejoras futuras se recomienda agregar efectos de sonido a las batallas para hacer la experiencia más inmersiva. También se podría implementar un sistema de niveles para los personajes donde suban sus estadísticas conforme ganan batallas. Finalmente se recomienda agregar animaciones a los personajes durante los ataques para mejorar la experiencia visual del juego.

Bibliografía

Python Software Foundation. (2024). Python Documentation. <https://docs.python.org>

Tkinter Documentation. (2024). Python Tkinter Reference.

<https://docs.python.org/3/library/tkinter.html>

Pillow Documentation. (2024). Pillow - Python Imaging Library.

<https://pillow.readthedocs.io>

